

CSI3007 - ADVANCED PYTHON PROGRAMMING
PYTHON MINI PROJECT
PRODUCT SALES TREND VIEWER

SUBMITTED BY (Group No - 22)

Preetha M 22MID0228

Haarini SK 22MID0231

Sivaranjani TS 22MID0232

Madhu Shree AN 22MID0284

Under the Supervision of

Dr. PRASANNA S

Professor

School of Computer Science and Engineering (SCOPE)
MTech (Integrated) in Computer Science and Engineering with
Specialization in Data Science School of Computer Science and
Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

October 2025

Datasets

- Default dataset: “sales_data.csv” with mapped columns to date, product, sales, and optional region/quantity/customer type per your app’s renaming logic.
- Uploaded dataset: “retail_sales_dataset.csv” with transaction-level fields (date, product category, quantity, unit price, total amount, customer demographics).

Key KPIs

- Total sales (default): Aggregates SalesAmount after cleansing and standardization to numeric, excluding rows with invalid dates or sales fields as your pipeline enforces.
- Average monthly sales: Mean of monthly summed sales using Period M grouping in your dashboard computation.
- Best month: Calendar month with max total sales identified via monthname grouping and idxmax.

Monthly trend and forecast

- Historical trend: Monthly aggregated line chart of Sales by month for selected products/region filters, reflecting seasonality and intra-year variance.
- 6-month forecast: Prophet model with yearly seasonality enabled and monthly freq; historical y overlaid with yhat projection for planning. A “not enough data” guard applies for sparse filters.

Top products

- Top 5 by sales: Ranked via product groupby sum descending; supports direct rendering as a bar chart and informs inventory allocation.
- Cross-check with input file: Product Category in retail dataset enables category-level comparisons when mapping to “product” semantics in your UI.

Region performance

- Region distribution: If region exists in mapped columns, the app renders a region pie chart for contribution analysis; otherwise falls back gracefully.
- Regional reps: Default file includes SalesRep and Region composites (“RegionandSalesRep”), aiding route and quota reviews when mapped.

Customer and channel mix

- Customer type: Optional “CustomerType” mapping (New vs Returning) enables cohort KPIs and retention tags in the filtered grid.
- Channels and payments: “SalesChannel” and “PaymentMethod” from default data support funnel and checkout preference insights.

Quantity, pricing, and discounts

- Volume: Optional “QuantitySold” mapping activates volume KPIs and price-volume checks in the filtered table.
- Price and discount: Default file’s UnitCost/UnitPrice and Discount fields allow margin diagnostics (e.g., high discount/low margin pockets).

- Input file totals: Retail dataset provides Quantity, Price per Unit, and Total Amount for basket-level validation of revenue.

Data quality notes

- Cleansing: Dates coerced to datetime; non-parsable dates and non-numeric Sales are dropped to protect KPIs.
- Renaming: Built-in mappings handle frequent schema variations (SaleDate→date, OrderDate→date, SalesAmount/SalesValue→sales, ProductName/Category→product).
- Edge values: The default CSV includes extreme/long floating values; numeric coercion and aggregation minimize distortion but warrant outlier review.

How to reproduce in your app

- Use default when no upload: The app auto-loads your default path and builds KPIs, Monthly Trend, Forecast, Top Products, and optional Region chart.
- Upload retail file: Map “Product Category→Product”, “Total Amount→Sales”, and “Date→Date”; optionally map “Quantity→Quantity Sold.” Click “Load and Analyze Data.”
- Filter: Apply product and region filters (if present). Download the filtered summary CSV from the provided export button.

Actionable insights framework

- Assortment: Increase stock and promo spend on top 5 products; validate cannibalization using month-on-month product lines.
- Regional targeting: Rebalance inventory and rep quotas to regions with rising share in the pie distribution.
- Retention: Tailor offers for Returning vs New cohorts using “CustomerType” to lift repeat rate without eroding margin.
- Promo efficiency: Use Discount vs Sales lift from default file to refine thresholds; validate in retail data with unit economics.
- Forecast planning: Commit inventory and targets to Prophet’s 6-month yhat with safety buffers where historical data is sparse post-filter.

Deliverables you can export now

- KPI snapshot: Total Sales, Avg Sales/Month, Best Month under current filters.sales_trend_viewer1.py
- Charts: Monthly Sales Trend and 6-Month Forecast; Top 5 Products bar; Region-wise pie (if region mapped).
- Filtered data table: Date, Region, Product, Sales, QuantitySold, CustomerType (superset of columns shown when present)
- CSV export: “Filtered Sales Data” download for sharing and audit.sales_trend_viewer1.py

CODE:

```
import pandas as pd

import plotly.express as px

import streamlit as st

from prophet import Prophet


st.set_page_config(

    page_title="💡 Product Sales Trend Viewer",

    page_icon=":bar_chart:",

    layout="wide"

)


# ===== APP HEADER WITH STYLING =====

st.markdown("""

<style>

    .main-title {font-family: 'Arial Black', sans-serif; color:#087EA4; font-size:2.4em;}

    .subtitle {color:#32337B; font-size:1.1em; font-weight:bold;}

    .sidebar-title {color:#09AB5D; font-size:1.1em;}

    div.stButton > button {background-color:#09AB5D; color:white; font-weight:bold;}

    div.stButton > button:hover {background-color:#087EA4;}

</style>

""", unsafe_allow_html=True)


st.markdown('<div class="main-title"> ⚡ Product Sales Trend Viewer</div>',

unsafe_allow_html=True)

st.markdown('<div class="subtitle">Easily upload any CSV, map columns, and explore product sales

with instant charts & forecasts!</div>', unsafe_allow_html=True)

st.write("")


# ===== SIDEBAR FOR UPLOAD AND MAPPING =====

st.sidebar.markdown('<div class="sidebar-title">Step 1: Upload and Map Columns</div>',

unsafe_allow_html=True)
```

```

uploaded_file = st.sidebar.file_uploader("Choose your sales CSV", type=['csv'])

# ----- DATA ACQUISITION -----

if uploaded_file:
    df_sample = pd.read_csv(uploaded_file, nrows=0)
    columns = df_sample.columns.tolist()
    st.sidebar.write("Detected columns:", columns)

    col_product = st.sidebar.selectbox(" ♦ Product column", columns, index=0)

    col_sales = st.sidebar.selectbox(" ♦ Sales/Amount column", columns, index=1 if len(columns)>1
    else 0)

    col_date = st.sidebar.selectbox(" ♦ Date column", columns, index=2 if len(columns)>2 else 0)

    optional_cols = columns + ["None"]
    col_region = col_quantity = col_customer = None

    if st.sidebar.checkbox("Map Region column (optional)"):
        col_region = st.sidebar.selectbox("Region column", optional_cols, index=len(optional_cols)-1)
        if col_region == "None": col_region = None

    if st.sidebar.checkbox("Map Quantity Sold column (optional)"):
        col_quantity = st.sidebar.selectbox("Quantity Sold column", optional_cols,
        index=len(optional_cols)-1)
        if col_quantity == "None": col_quantity = None

    if st.sidebar.checkbox("Map Customer Type column (optional)"):
        col_customer = st.sidebar.selectbox("Customer Type column", optional_cols,
        index=len(optional_cols)-1)
        if col_customer == "None": col_customer = None

    analyze_clicked = st.sidebar.button(" 🚀 Load and Analyze Data", use_container_width=True)

# Read and Process Data if Button Is Clicked

if analyze_clicked:
    with st.spinner("Processing CSV and generating dashboard..."):

```

```

uploaded_file.seek(0)
df = pd.read_csv(uploaded_file)
rename_map = {col_product: 'product', col_sales: 'sales', col_date: 'date'}
if col_region: rename_map[col_region] = 'region'
if col_quantity: rename_map[col_quantity] = 'quantity_sold'
if col_customer: rename_map[col_customer] = 'customer_type'
df = df.rename(columns=rename_map)
df['date'] = pd.to_datetime(df['date'], errors='coerce')
df = df.dropna(subset=['date'])
df['sales'] = pd.to_numeric(df['sales'], errors='coerce')
df = df.dropna(subset=['sales'])
if 'region' in df.columns:
    df['region'] = df['region'].astype(str)
if 'quantity_sold' in df.columns:
    df['quantity_sold'] = pd.to_numeric(df['quantity_sold'], errors='coerce')
if 'customer_type' in df.columns:
    df['customer_type'] = df['customer_type'].astype(str)
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month_name()

```

else:

```

# ----- FALLBACK TO DEFAULT CSV -----
DATA_PATH = "C:/Users/PREETHA/Downloads/sales_data.csv"
st.sidebar.write("No file uploaded. Using default CSV.")
@st.cache_data
def load_data():
    df = pd.read_csv(DATA_PATH)
    df.columns = df.columns.str.strip().str.lower()
    df = df.rename(columns={
        'sale_date': 'date',
        'order_date': 'date',
        'sales_amount': 'sales',

```

```

        'sales_value': 'sales',
        'product_category': 'product',
        'product_name': 'product'
    })
    df['date'] = pd.to_datetime(df['date'], errors='coerce')
    df = df.dropna(subset=['date'])
    df['year'] = df['date'].dt.year
    df['month'] = df['date'].dt.month_name()
    return df
try:
    df = load_data()
except Exception as e:
    st.error(f"❌ Error loading file: {e}")
    st.stop()

# ===== DASHBOARD + FILTERS =====
if (uploaded_file and analyze_clicked) or not uploaded_file:
    st.sidebar.markdown('<div class="sidebar-title">Step 2: Filter Options</div>',
unsafe_allow_html=True)
    product_list = sorted(df['product'].dropna().unique())
    region_list = ['All'] + sorted(df['region'].dropna().unique()) if 'region' in df.columns else ['All']
    selected_products = st.sidebar.multiselect("Select Product(s)", product_list,
default=product_list[:1])
    selected_region = st.sidebar.selectbox("Select Region", region_list)

    filtered_df = df[df['product'].isin(selected_products)]
    if selected_region != 'All' and 'region' in df.columns:
        filtered_df = filtered_df[filtered_df['region'] == selected_region]

    if filtered_df.empty:
        st.warning("No sales data available for your filter selection.")
        st.stop()

```

```

st.markdown("---")

st.markdown('<span class="subtitle">Key Performance Indicators</span>',
unsafe_allow_html=True)

total_sales = filtered_df['sales'].sum()

monthly_sales_mean =
filtered_df.groupby(filtered_df['date'].dt.to_period('M'))['sales'].sum().mean()

best_month = filtered_df.groupby(filtered_df['date'].dt.month_name())['sales'].sum().idxmax()

col1, col2, col3 = st.columns(3)

col1.metric("Total Sales", f"{total_sales:,.0f}")

col2.metric("Avg Sales/Month", f"{monthly_sales_mean:,.0f}")

col3.metric("Best Month", best_month)


st.markdown("---")

st.subheader("📅 Monthly Sales Trend & Forecast")

monthly_sales =
filtered_df.groupby(filtered_df['date'].dt.to_period('M'))['sales'].sum().reset_index()

monthly_sales['date'] = monthly_sales['date'].dt.to_timestamp()

fig1 = px.line(monthly_sales, x='date', y='sales', markers=True, title="Historic Monthly Sales")

st.plotly_chart(fig1, use_container_width=True)


st.markdown("***Sales Forecast for Next 6 Months***")

forecast_window = 6

prophet_df = monthly_sales.rename(columns={'date': 'ds', 'sales': 'y'})

valid_rows = prophet_df.dropna(subset=['ds', 'y']).shape[0]

if valid_rows < 2:

    st.warning("Not enough data after filtering for forecasting. Please select more data or check your mapped columns.")

else:

    m = Prophet(yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False)

    m.fit(prophet_df)

    future = m.make_future_dataframe(periods=forecast_window, freq='M')

    forecast = m.predict(future)

    fig2 = px.line(

        forecast, x='ds', y='yhat', title="Forecasted Sales",

```



```

        labels={'ds': 'Month', 'yhat': 'Sales Forecast'}
    )
    fig2.add_scatter(x=prophet_df['ds'], y=prophet_df['y'], mode='markers', name='Historical')
    st.plotly_chart(fig2, use_container_width=True)

st.markdown("---")
st.subheader("🏆 Top 5 Products by Total Sales")
top_products = (
    df.groupby('product')['sales']
    .sum()
    .sort_values(ascending=False)
    .head(5)
)
st.bar_chart(top_products)

if 'region' in df.columns:
    st.markdown("---")
    st.subheader("🌍 Region-wise Sales Distribution")
    region_sales = df.groupby('region')['sales'].sum().reset_index()
    fig3 = px.pie(region_sales, names='region', values='sales', title="Sales by Region")
    st.plotly_chart(fig3, use_container_width=True)

st.markdown("---")
st.subheader("📋 Filtered Data Summary")

show_cols = [col for col in ['date', 'region', 'product', 'sales', 'quantity_sold', 'customer_type'] if col
in df.columns]

st.dataframe(filtered_df[show_cols].sort_values('date', ascending=False))

def convert_df_to_csv(df):
    return df.to_csv(index=False).encode('utf-8')

csv_data = convert_df_to_csv(filtered_df[show_cols])

st.download_button(

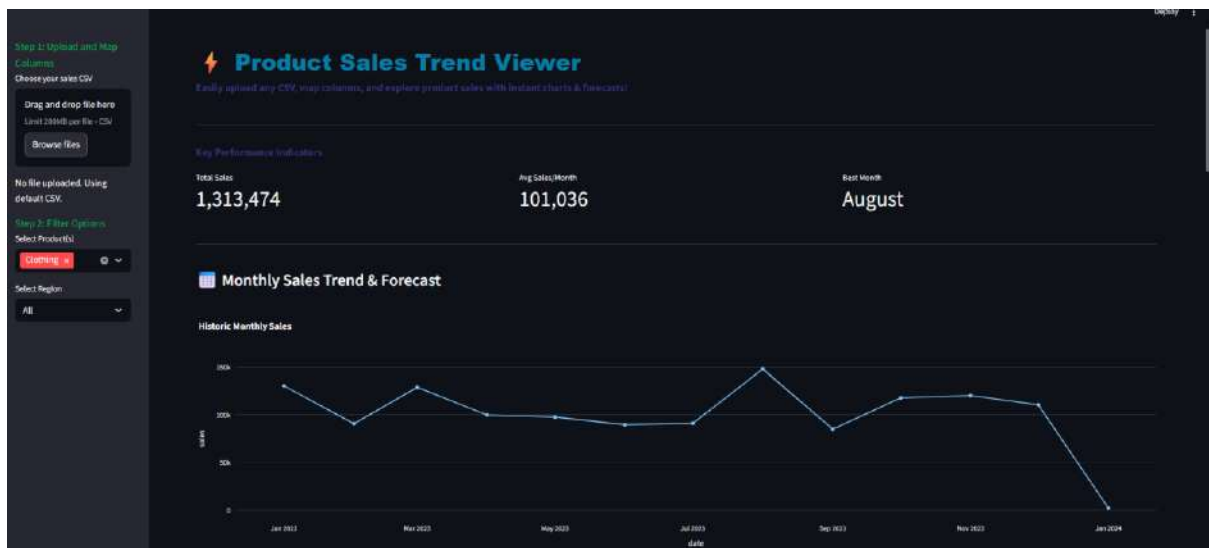
```

```

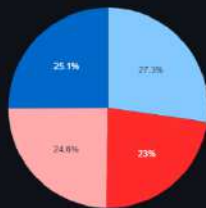
label="⬇️ Download Filtered Data as CSV",
data=csv_data,
file_name='filtered_sales_data.csv',
mime='text/csv'
)
else:
    st.info("Please upload a CSV and click 'Load and Analyze Data' or use the default CSV for instant dashboard!")

```

OUTPUT:



Sales by Region



No file uploaded. Using default CSV.

Step 2: Filter Options

Select Products

Clothing x

Food x

Select Region

North

Filtered Data Summary

	date	region	product	sales	quantity_sold	customer_type
619	2023-12-25 00:00:00	North	Clothing		2836.04	26 Returning
71	2023-12-22 00:00:00	North	Clothing		8485.9	11 New
723	2023-12-22 00:00:00	North	Clothing		1940.17	45 Returning
692	2023-12-20 00:00:00	North	Clothing		2782.33	2 New
107	2023-12-19 00:00:00	North	Food		5094.54	21 New
323	2023-12-15 00:00:00	North	Food		3807.13	32 New
710	2023-12-13 00:00:00	North	Clothing		3736.89	16 New
374	2023-12-12 00:00:00	North	Clothing		2475.95	8 New
729	2023-12-10 00:00:00	North	Food		9989.64	7 Returning
381	2023-12-08 00:00:00	North	Food		7567.06	22 New

Download Filtered Data as CSV